

|                                                                                                             |                                                                                                                            |                                 |
|-------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|---------------------------------|
|  <b>Marwadi University</b> | <b>Marwadi University</b><br><b>Faculty of Technology</b><br><b>Department of Information and Communication Technology</b> |                                 |
| <b>Subject: Artificial Intelligence (01CT0616)</b>                                                          | <b>Aim:</b> To recommend an item to the user using content-based filtering approach                                        |                                 |
| <b>Experiment No: 11</b>                                                                                    | <b>Date:</b>                                                                                                               | <b>Enrolment No:92301733022</b> |

**Aim:** To recommend an item to the user using content-based filtering approach

**IDE:** Google Colab

### **Theory:**

There are a lot of applications where websites collect data from their users and use that data to predict the likes and dislikes of their users. This allows them to recommend the content that they like. Recommender systems are a way of suggesting or similar items and ideas to a user's specific way of thinking.

Recommender System is different types:

- **Collaborative Filtering:** Collaborative Filtering recommends items based on similarity measures between users and/or items. The basic assumption behind the algorithm is that users with similar interests have common preferences.
- **Content-Based Recommendation:** It is supervised machine learning used to induce a classifier to discriminate between interesting and uninteresting items for the user.

**Content-Based Recommendation System:** Content-Based systems recommends items to the customer similar to previously high-rated items by the customer. It uses the features and properties of the item. From these properties, it can calculate the similarity between the items.

In a content-based recommendation system, first, we need to create a profile for each item, which represents the properties of those items. From the user profiles are inferred for a particular user. We use these user profiles to recommend the items to the users from the catalog.

#### **Item profile:**

In a content-based recommendation system, we need to build a profile for each item, which contains the important properties of each item. For Example, If the movie is an item, then its actors, director, release year, and genre are its important properties, and for the document, the important property is the type of content and set of important words in it.

Let's have a look at how to create an item profile. First, we need to perform the TF-IDF vectorizer, here TF (term frequency) of a word is the number of times it appears in a document and The IDF (inverse document frequency) of a word is the measure of how significant that term is in the whole corpus.

#### **User profile:**

The user profile is a vector that describes the user preference. During the creation of the user's profile, we use a utility matrix that describes the relationship between user and item. From this information, the best estimate we can decide which item the user likes, is some aggregation of the profiles of those items.

#### **Advantages and Disadvantages:**

- **Advantages:**
  - No need for data on other users when applying to similar users.
  - Able to recommend to users with unique tastes.
  - Able to recommend new & popular items

|                                                                                                             |                                                                                                                            |                                 |
|-------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|---------------------------------|
|  <b>Marwadi University</b> | <b>Marwadi University</b><br><b>Faculty of Technology</b><br><b>Department of Information and Communication Technology</b> |                                 |
| <b>Subject: Artificial Intelligence (01CT0616)</b>                                                          | <b>Aim:</b> To recommend an item to the user using content-based filtering approach                                        |                                 |
| <b>Experiment No: 11</b>                                                                                    | <b>Date:</b>                                                                                                               | <b>Enrolment No:92301733022</b> |

- Explanations for recommended items.
- **Disadvantages:**
  - Finding the appropriate feature is hard.
  - Doesn't recommend items outside the user profile.

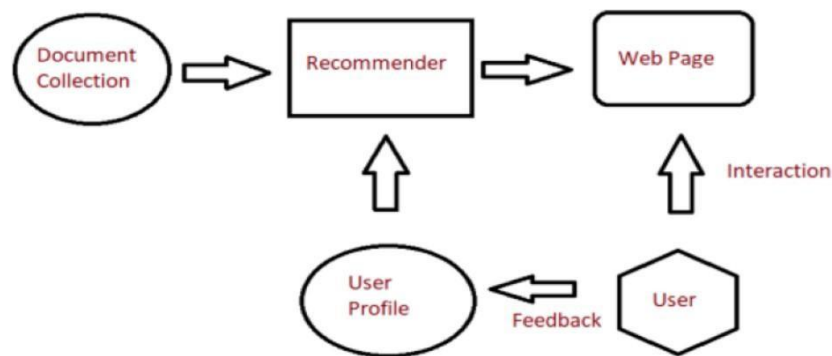


Fig: Recommender System

### Pre Lab Exercise:

1. When can you use content-based recommendation system approach?

Content-based recommendation is used when we know item features such as genre, category, actor, brand, keywords, etc. It recommends similar items based on user past likes. Example: movie, news, product recommendation.

2. Write advantages of content-based filtering approach

- Recommends items based on user interest
- No need data of other users
- Can recommend new items
- Good for unique users
- Fast and personalized suggestions

3. Write disadvantages of content-based filtering approach

- Need proper item features
- Cannot recommend very different items
- Limited discovery
- User profile may become narrow

|                                                                                                                |                                                                                                                            |                                 |
|----------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|---------------------------------|
|  <b>Marwadi</b><br>University | <b>Marwadi University</b><br><b>Faculty of Technology</b><br><b>Department of Information and Communication Technology</b> |                                 |
| <b>Subject: Artificial Intelligence (01CT0616)</b>                                                             | <b>Aim:</b> To recommend an item to the user using content-based filtering approach                                        |                                 |
| <b>Experiment No: 11</b>                                                                                       | <b>Date:</b>                                                                                                               | <b>Enrolment No:92301733022</b> |

### **Program (Code):**

```
!pip install opendatasets
import opendatasets as od
```

```
dataset_url = 'https://www.kaggle.com/datasets/harshshinde8/movies-csv/data'
od.download(dataset_url)
```

```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
```

```
movies = pd.read_csv('/content/movies-csv/movies.csv')
# Preprocessing step
movies["genres"] = movies["genres"].fillna("")
movies["overview"] = movies["overview"].fillna("")
movies["keywords"] = movies["keywords"].fillna("")
# TF-IDF Vectorizer
tfidf = TfidfVectorizer(stop_words='english')
tfidf_matrix = tfidf.fit_transform(movies['keywords'])
cosine_sim = cosine_similarity(tfidf_matrix, tfidf_matrix)
# Dropping the duplicates
indices = pd.Series(movies.index, index=movies['original_title']).drop_duplicates()
def recommendation_movie(title, topn = 5):
    if title not in indices:
        return "Movie not found!"
    idx = indices[title]
    sim_scores = list(enumerate(cosine_sim[idx]))
    sim_scores = sorted(sim_scores, key=lambda x: x[1], reverse=True)
    sim_scores = sim_scores[1:topn+1]
    movie_indices = [i[0] for i in sim_scores]
    return movies['original_title'].iloc[movie_indices]
recommendation_movie("The Dark Knight")
```

### **Results:**

|                                                                                                                |                                                                                                                            |                                 |
|----------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|---------------------------------|
|  <b>Marwadi</b><br>University | <b>Marwadi University</b><br><b>Faculty of Technology</b><br><b>Department of Information and Communication Technology</b> |                                 |
| <b>Subject: Artificial Intelligence (01CT0616)</b>                                                             | <b>Aim:</b> To recommend an item to the user using content-based filtering approach                                        |                                 |
| <b>Experiment No: 11</b>                                                                                       | <b>Date:</b>                                                                                                               | <b>Enrolment No:92301733022</b> |

```

...
original_title
3    The Dark Knight Rises
119   Batman Begins
1359  Batman
210   Batman & Robin
428   Batman Returns

dtype: object

```

### Observation:

1. Better metadata gives better results.
2. Personalized output generated.
3. Works well for movies/books/products.
4. Recommendations depend on item similarity.

### Post Lab Exercise:

1. What changes you feel can be done, to improve the system that you have designed, as a part of in-house lab exercise.
  - Add user ratings
  - Use hybrid recommendation
  - Add trending items
  - Better feature extraction
  - Use deep learning embeddings
2. Recommend top-10 “series” using IMDB movie dataset

|                                                                                                             |                                                                                                                            |                                 |
|-------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|---------------------------------|
|  <b>Marwadi University</b> | <b>Marwadi University</b><br><b>Faculty of Technology</b><br><b>Department of Information and Communication Technology</b> |                                 |
| <b>Subject: Artificial Intelligence (01CT0616)</b>                                                          | <b>Aim:</b> To recommend an item to the user using content-based filtering approach                                        |                                 |
| <b>Experiment No: 11</b>                                                                                    | <b>Date:</b>                                                                                                               | <b>Enrolment No:92301733022</b> |

(Link: <https://www.kaggle.com/datasets/harshitshankhdhar/imdb-dataset-of-top-1000-movies-and-tv-shows>). Attach the code with output. Also, write your observation for the same.

Code:

```
# Filter for series (assuming 'TV Series' or similar in Genre column)
series_df = imdb_df[imdb_df['Genre'].str.contains('TV Series|Miniseries', case=False, na=False)]

if not series_df.empty:
    # Sort by IMDB_Rating in descending order and get the top 10
    top_10_series = series_df.sort_values(by='IMDB_Rating', ascending=False).head(10)
    print("Top 10 Series based on IMDB Rating (filtered by 'TV Series|Miniseries' in Genre):")
    print(top_10_series[['Series_Title', 'IMDB_Rating', 'Genre', 'Released_Year']])
    print("\nObservation: The dataset contains a mix of movies and some highly-rated limited series or miniseries. This filtering successfully identified and ranked these based on their IMDB Rating.")
else:
    print("No series explicitly identified by 'TV Series' or 'Miniseries' keywords in the 'Genre' column.")
    print("The filtering based on these genre keywords resulted in an empty DataFrame.")
    print("Further investigation into other columns (like Runtime or broader genre terms) is needed to identify series, or the dataset might predominantly contain movies categorized without explicit 'series' labels.")
```

Output:

```
No series explicitly identified by 'TV Series' or 'Miniseries' keywords in the 'Genre' column.
The filtering based on these genre keywords resulted in an empty DataFrame.
Further investigation into other columns (like Runtime or broader genre terms) is needed to identify series, or the dataset might predominantly contain movies categorized without explicit 'series' labels.
```